

LVN Finder

//

// LVN Finder

// Version: 1.0

// Author: QuantShok (JacobS369)

//

// This script is open source. If you use, modify, or redistribute this code

// (in whole or in part), you must credit the original author:

//

// Original script by QuantShok (JacobS369)

// <https://www.tradingview.com/u/JacobS369/>

//

// Default settings are optimized for NQ (Nasdaq 100 Futures) on the 5m chart.

//

//@version=6

indicator("LVN Finder (5m Chart)", shorttitle="LVN-Finder", overlay=true)

//

// VOLUME PROFILE SETTINGS

//

var string G_VP = " Volume Profile Settings"

vp_rows = input.int(10000, "Number of Rows", minval=50, maxval=10000, step=10, group=G_VP)

lvn_lookback_days = input.int(5, "LVN Lookback Days", minval=1, maxval=20, step=1, group=G_VP)

lvn_rebuild_interval_days = input.int(1, "Rebuild Interval (days)", minval=1, maxval=10, step=1, group=G_VP)

```

var string G_EXTREMA = "🔍 LVN Detection"
vp_window_size_pct   = input.float(1.0, "Window Size %",   minval=1.0, maxval=20.0, step=0.5,
group=G_EXTREMA)
vp_lvn_prominence    = input.float(10.00, "LVN Prominence %", minval=5.0, maxval=50.0,
step=1.0, group=G_EXTREMA)
vp_use_lvn_min_vol    = input.bool(true, "Use LVN Min Volume Filter", inline="lvn_min",
group=G_EXTREMA)
vp_lvn_min_volume_pct = input.float(5.0, "LVN Min %",        minval=0.0, maxval=50.0, step=1.0,
inline="lvn_min", group=G_EXTREMA)

var string G_LVN = "🟦 LVN Display"
vp_show_lvn = input×bool(true,      "Show LVNs", inline="lvn_show", group=G_LVN)
vp_lvn_color = input.color(color.blue, "Color",    inline="lvn_show", group=G_LVN)
vp_lvn_width = input.int(2,          "Line Width", minval=1, maxval=10, group=G_LVN)

var string G_MERGE = "⚙️ Level Management"
vp_merge_enabled = input.bool(true, "Merge Nearby Levels", group=G_MERGE)
vp_merge_ticks   = input.int(20,    "Merge Distance (ticks)", minval=1, maxval=1000,
group=G_MERGE)
vp_merge_pct     = input.float(0.01, "Merge Distance (%)", minval=0.01,maxval=2.0, step=0.01,
group=G_MERGE)
vp_extendBars    = input.int(100,    "Extend Lines (bars)", minval=10, maxval=500, step=10,
group=G_MERGE)
vp_limit_lvn_count = input.bool(true, "Limit LVN Count",      inline="lvn_limit", group=G_MERGE)
vp_max_lvn_count   = input.int(15,    "Max",                  minval=1, maxval=50, inline="lvn_limit",
group=G_MERGE)

//
//
// TIME-BASED LOOKBACK
//
//

calculate_bars_for_days(int target_days) =>
    int bars_found          = 0
    int trading_days_found  = 0

```

```

int current_day_timestamp = -1
for i = 0 to bar_index
    int bar_day_id = year(time[i]) * 10000 + month(time[i]) * 100 + dayofmonth(time[i])
    if bar_day_id != current_day_timestamp
        trading_days_found += 1
        current_day_timestamp := bar_day_id
        if trading_days_found > target_days
            break
    bars_found += 1
bars_found

is_new_trading_day(int last_rebuild_time) =>
    bool result = false
    if last_rebuild_time == 0
        result := true
    else
        int current_day_id = year(time) * 10000 + month(time) * 100 +
dayofmonth(time)
        int last_day_id = year(last_rebuild_time) * 10000 + month(last_rebuild_time) * 100 +
dayofmonth(last_rebuild_time)
        result := current_day_id != last_day_id
    result

```

```
//
```

```
// VOLUME PROFILE CALCULATION
```

```
//
```

```

var array<float> profile_prices = array.new<float>()
var array<float> profile_volumes = array.new<float>()
var array<label> lvn_labels = array.new<label>()
var array<float> lvn_zone_centers = array.new<float>()
var array<int> lvn_zone_row_indices = array.new<int>()
var array<line> lvn_lines = array.new<line>()

```

```

build_vp(int lookbackBars) =>
    profile_prices.clear()
    profile_volumes.clear()
    [lower_tf_high, lower_tf_low, lower_tf_volume] = request.security(syminfo.tickerid, "1", [high, low,
volume])
    int max_lower_tf_lookback = 50
    int actual_lookback    = math×min(lookbackBars, bar_index + 1)
    if actual_lookback >= 100
        float range_high  = math.max(ta.highest(high, actual_lookback), ta.highest(lower_tf_high,
math.min(actual_lookback, max_lower_tf_lookback)))
        float range_low   = math.min(ta.lowest(low, actual_lookback), ta.lowest(lower_tf_low,
math.min(actual_lookback, max_lower_tf_lookback)))
        float range_height = range_high - range_low
        if range_height > 0
            float row_height    = range_height / vp_rows
            float half_row_height = row_height / 2
            for i = 0 to vp_rows - 1
                profile_prices.push(range_low + row_height * (i + 0.5))
                profile_volumes.push(0.0)
            for bar_back = 0 to actual_lookback - 1
                bool use_lower_tf = bar_back < max_lower_tf_lookback
                float bar_high  = use_lower_tf ? lower_tf_high[bar_back] : high[bar_back]
                float bar_low   = use_lower_tf ? lower_tf_low[bar_back]  : low[bar_back]
                float bar_vol   = use_lower_tf ? lower_tf_volume[bar_back] : volume[bar_back]
                float bar_range = bar_high - bar_low
                if bar_range <= 0
                    continue
                int first_row    = math×max(0, math.min(vp_rows - 1, int((bar_low - range_low) /
row_height)))
                int last_row     = math×max(0, math.min(vp_rows - 1, int((bar_high - range_low) /
row_height)))
                float inv_bar_range = 1.0 / bar_range
                for row_idx = first_row to last_row
                    float row_price = profile_prices.get(row_idx)
                    float overlap   = math.min(bar_high, row_price + half_row_height) - math.max(bar_low,
row_price - half_row_height)
                    profile_volumes.set(row_idx, profile_volumes.get(row_idx) + bar_vol * overlap *
inv_bar_range)

```

```
//
```

```
// LVN DETECTION & FILTERING
```

```
//
```

```
detect_lvn_valleys() =>
    array<int> valleys = array.new<int>()
    if profile_volumes.size() >= 3
        int window_size = int(vp_rows * vp_window_size_pct / 100.0)
        float max_volume = array.max(profile_volumes)
        float min_abs_volume = vp_use_lvn_min_vol ? max_volume * vp_lvn_min_volume_pct / 100.0 :
0.0
        int edge_buffer = int(vp_rows * 0.05)
        for i = window_size + edge_buffer to vp_rows - window_size - edge_buffer - 1
            float current_vol = profile_volumes.get(i)
            if current_vol < min_abs_volume
                continue
            bool is_local_min = true
            for offset = 1 to window_size
                if profile_volumes.get(math.max(0, i - offset)) < current_vol or
profile_volumes.get(math.min(vp_rows - 1, i + offset)) < current_vol
                    is_local_min := false
                    break
            if not is_local_min
                continue
            float left_max = current_vol
            float right_max = current_vol
            for j = i - 1 to math.max(0, i - 100)
                float v = profile_volumes.get(j)
                if v > left_max
                    left_max := v
                if v < current_vol
                    break
            for j = i + 1 to math.min(vp_rows - 1, i + 100)
```

```

float v = profile_volumes.get(j)
if v > right_max
    right_max := v
if v < current_vol
    break
float ceiling = math.min(left_max, right_max)
float prominence_pct = (ceiling - current_vol) / ceiling * 100.0
if prominence_pct >= vp_lvn_prominence
    valleys.push(i)
valleys

```

```

calculate_lvn_zones(array<int> valley_indices) =>
    lvn_zone_centers.clear()
    lvn_zone_row_indices.clear()
    if valley_indices.size() > 0
        for i = 0 to valley_indices.size() - 1
            int valley_row = valley_indices.get(i)
            float center_price = profile_prices.get(valley_row)
            lvn_zone_centers.push(center_price)
            lvn_zone_row_indices.push(valley_row)

```

```

merge_nearby_levels(array<int> indices_array) =>
    array<int> merged = array.new<int>()
    if vp_merge_enabled and indices_array.size() > 1
        merged.push(indices_array.get(0))
        for i = 1 to indices_array.size() - 1
            int current_idx = indices_array.get(i)
            float current_price = profile_prices.get(current_idx)
            float current_vol = profile_volumes.get(current_idx)
            bool should_add = true
            for j = 0 to merged.size() - 1
                int existing_idx = merged.get(j)
                float existing_price = profile_prices.get(existing_idx)
                float existing_vol = profile_volumes.get(existing_idx)
                float tick_dist = math.abs(current_price - existing_price) / syminfo.mintick
                float pct_dist = math.abs(current_price - existing_price) / current_price * 100.0
                if tick_dist <= vp_merge_ticks or pct_dist <= vp_merge_pct
                    should_add := false

```

```

        if current_vol < existing_vol
            merged.set(j, current_idx)
        break
    if should_add
        merged.push(current_idx)
else
    merged := indices_array.copy()
merged

```

```

filter_top_n_lvn(array<int> indices_array, int max_count) =>
    array<int> result = array.new<int>()
    if not vp_limit_lvn_count or indices_array.size() <= max_count
        result := indices_array.copy()
    else
        array<int> sorted = indices_array.copy()
        int items = math.min(max_count, sorted.size())
        for i = 0 to items - 1
            int min_idx = i
            float min_vol = profile_volumes.get(sorted.get(i))
            for j = i + 1 to sorted.size() - 1
                float v = profile_volumes.get(sorted.get(j))
                if v < min_vol
                    min_idx := j
                    min_vol := v
            if min_idx != i
                int temp = sorted.get(i)
                sorted.set(i, sorted.get(min_idx))
                sorted.set(min_idx, temp)
        for i = 0 to items - 1
            result.push(sorted.get(i))
    result

```

```
//
```

```
// DRAWING
```

```
//
```

```

draw_lv_n_zones(int start_bar) =>
    if lv_n_lines.size() > 0
        for i = 0 to lv_n_lines.size() - 1
            line.delete(lv_n_lines.get(i))
        lv_n_lines.clear()
    if lv_n_labels.size() > 0
        for i = 0 to lv_n_labels.size() - 1
            label.delete(lv_n_labels.get(i))
        lv_n_labels.clear()
    if vp_show_lv_n and lv_n_zone_centers.size() > 0
        for i = 0 to lv_n_zone_centers.size() - 1
            float center = lv_n_zone_centers.get(i)
            int x1 = time[bar_index - start_bar]
            int x2 = time + vp_extend_bars * (time - time[1])
            line l = line.new(x1, center, x2, center, xloc=xloc.bar_time, color=vp_lv_n_color,
width=vp_lv_n_width, style=line.style_solid, extend=extend.right)
            lv_n_lines.push(l)
            label lbl = label.new(x1, center, str.tostring(center, "###"),
xloc=xloc.bar_time,
style=label.style_label_left,
color=vp_lv_n_color,
textcolor=color.white,
size=size_x_small)
            lv_n_labels.push(lbl)

```

```
//
```

```
// MAIN REBUILD LOGIC
```

```
//
```

```

var int last_rebuild_time      = 0
var int rebuild_start_bar     = 0
var int trading_days_since_rebuild = 0

```

```

bool new_day = is_new_trading_day(last_rebuild_time)
if new_day and bar_index > 0
    trading_days_since_rebuild += 1

bool should_rebuild = bar_index == 0 or (new_day and trading_days_since_rebuild >=
lvn_rebuild_interval_days)
if should_rebuild
    int lookbackBars = calculate_bars_for_days(lvn_lookback_days)
    build_vp(lookbackBars)
    array<int> lvn_raw = detect_lvn_valleys()
    array<int> lvn_merged = merge_nearby_levels(lvn_raw)
    lvn_merged := filter_top_n_lvn(lvn_merged, vp_max_lvn_count)
    calculate_lvn_zones(lvn_merged)
    trading_days_since_rebuild := 0
    last_rebuild_time := time
    rebuild_start_bar := bar_index

draw_lvn_zones(rebuild_start_bar)

//

```

```

// LVN LEVEL TABLE
//

```

```

var table lvn_table = table.new(position.top_right, 2, 16,
    border_width = 2,
    border_color = color.blue,
    frame_width = 2,
    frame_color = color×blue)

if barstate.islast and lvn_zone_centers.size() > 0
    table.cell(lvn_table, 0, 0, "LVN #", bgcolor=color.new(color.blue, 0), text_color=color×white,
text_size=size×small)
    table.cell(lvn_table, 1, 0, "Price", bgcolor=color.new(color.blue, 0), text_color=color×white,

```

```
text_size=size×small)
for i = 0 to math.min(14, lvn_zone_centers.size() - 1)
    float lvn_price = lvn_zone_centers.get(i)
    table.cell(lvn_table, 0, i + 1, str.toString(i + 1),
        bgcolor=color.new(color.blue, 80),
        text_color=color×white,
        text_size=size×small)
    table.cell(lvn_table, 1, i + 1, str.toString(lvn_price, "#.##"),
        bgcolor=color.new(color.blue, 80),
        text_color=color×white,
        text_size=size.normal)
```